

PHASE 2

CREATING A CUSTOM TABLE AND FIELDS

Designing the data model that stores every employee-raised issue, from short description through resolution.

Employee Raise Issue — ServiceNow Implementation Guide
Standalone Phase Document 2 of 7

Document Version 1.0

About This Document

This document is a standalone, self-contained implementation guide for Phase 2 — Creating a Custom Table and Fields — of the Employee Raise Issue project. It can be read and actioned on its own, without needing the full multi-phase master documentation, though it references the wider project for context where useful.

Field	Detail
Application	Employee Raise Issue (built within the Employee Center scope)
Phase	Phase 2 of 7
Primary Owner	Platform / ServiceNow Administrator
Related Document	Employee Raise Issue Documentation (full 7-phase project reference)
Instance Referenced	dev218486.service-now.com
Document Version	1.0

Contents

1. Purpose and Scope
2. Where This Phase Fits in the Project
3. Objective
4. Prerequisites
5. Creating the Table
6. Field Design
7. Screenshot Reference
8. Choice List Values
 - 8.1 Category
 - 8.2 Subcategory (dependent on Category)
9. Table-Level Configuration
10. Access Control (ACL) Overview
11. Data Quality Considerations
 - 11.1 Sample Record
12. Common Issues and Troubleshooting
13. Validating Phase 2 Completion
14. Best Practices
 - 14.1 Frequently Asked Questions
15. Summary and Handoff to Phase 3
 - 15.1 Phase 2 Deliverables Checklist

1. Purpose and Scope

This document is the complete, standalone implementation guide for Phase 2 of the Employee Raise Issue project: designing and building the custom table that stores every issue an employee raises. It assumes Phase 1 (Creating a Custom Application) is complete and the target application scope already exists.

By the end of this phase, a fully fielded Employee Issue table exists, with the choice lists, dependent values, and table-level settings needed to support the UI Policies, Record Producer, and portal experience built in later phases.

2. Where This Phase Fits in the Project

Phase 2 turns the empty application container from Phase 1 into an actual data model. Everything downstream depends on the fields defined here:

- Phase 3's UI Policies reference these exact fields to control visibility, mandatory state, and dependency.
- Phase 4's Record Producer maps its catalog variables onto these fields.
- Phase 6's My Issues-style widgets query this table directly to show employees their submission history.
- Phase 7's test cases are written against the validation rules defined on these fields.

WATCH OUT: *Because so much depends on this table, changes to field names or types after Phase 4 onward are more expensive — Record Producer variable mappings, UI Policies, and widget server scripts all need to be revisited. Get the field list right in this phase where possible.*

3. Objective

Define a single custom table, Employee Issue, that stores every issue raised by an employee, along with the fields required to categorize, prioritize, route, and eventually resolve it. The table should be usable both by employees (via the Record Producer and portal) and by support agents (via a standard or Agent Workspace form).

4. Prerequisites

- Phase 1 complete: a scoped application exists and is selected as the active scope.
- Agreement on the initial field list and choice values with HR and IT Support stakeholders, since these define what employees will be asked to provide.
- A decision on whether the table should extend Task (inheriting Number, State, Priority, Assignment Group) or stand alone as a lighter-weight base table.

5. Creating the Table

1. From within Studio (with the correct application scope active), select Create Application File, then choose Table.
2. Set the Label to Employee Issue (or, as in the working build, Employee raises issue).
3. Confirm the auto-generated Name matches your naming convention, e.g. x_snc_emp_issue_issue or sn_ex_sp_employee_issue.
4. Decide whether to set Extends table to Task. Extending Task gives you Number, State, Priority, and Assignment Group fields immediately, at the cost of also inheriting many Task fields you may not need. Leaving it blank gives a lighter, purpose-built table.
5. Enable Create module and Create mapped Access Control Rules so that a navigator module and default table-level ACLs are generated automatically.

6. Select Auto Number for the Number field so every issue record receives a unique, human-readable identifier (for example EMP0000010).
7. Save the table definition.
8. Confirm the new table appears under the application's Tables list in Studio, and that its Application field shows the correct scope.

The screenshot displays the 'Employee raises issue' table definition in Studio. The interface includes a top bar with navigation icons and buttons for 'Delete', 'Update', and 'Delete All Records'. A notification bar at the top indicates 'Added new column Number'. The main form contains several sections: 'Label' with the value 'Employee raises issue', 'Name' with the value 'sn_ex_sp_employee_raises_issue', and 'Extends table' with a search icon. The 'Application' section shows 'Employee Center' selected. The 'Create module' and 'Create mobile module' checkboxes are both checked. The 'Add module to menu' dropdown is set to '-- Create new --'. The 'New menu name' field is empty. The 'Remote Table' checkbox is unchecked. At the bottom, the 'Columns' tab is selected, showing 'Extensible' and 'Live feed' options.

Figure 2 — Employee raises issue table definition (sn_ex_sp_employee_issue) as created in the Employee Center application, with Create module and Create mobile module enabled.

NOTE: The table was labeled *Employee raises issue* (technical name *sn_ex_sp_employee_issue*) in the working build. This document refers to it as *Employee Issue* throughout for readability; treat the two as the same table when following these steps against the live instance.

6. Field Design

The following fields were added to the Employee Issue table. Field names and types were chosen to support both the Record Producer intake experience (Phase 4) and downstream reporting.

Field Label	Type	Mandatory	Purpose
Short Description	String (160)	Yes	One-line summary of the issue shown in lists and notifications.
Description	String (Full UTF-8 Text)	Yes	Full details of the issue as provided by the employee.
Category	Choice	Yes	Top-level classification, e.g. Software, Hardware, Network, Access.

Field Label	Type	Mandatory	Purpose
Subcategory	Choice (dependent)	Conditional	Second-level classification, dependent on Category.
Priority	Choice	Yes	Employee-perceived urgency: Low, Medium, High, Critical.
Requester	Reference (sys_user)	Yes	The employee raising the issue; defaults to the submitting user.
Department	Reference (cmn_department)	No	Derived from the Requester for routing and reporting.
Preferred Contact Method	Choice	No	Email, Phone, or Portal Message.
Supporting Attachment	Attachment	No	Optional screenshot or supporting document.
State	Choice	System-managed	New, In Progress, Pending Employee, Resolved, Closed.
Assignment Group	Reference (sys_user_group)	System-managed	Set based on Category once routing is configured.
Resolution Notes	String (Full UTF-8 Text)	No	Populated by the assigned agent on resolution.

The screenshot shows a web form titled "Employee raises issue" with a "New record" link. The form contains the following fields:

- Number:** EMP0000010
- * Requester:** A text input field with a search icon.
- Category:** A dropdown menu showing "-- None --".
- Subcategory:** A dropdown menu showing "-- None --".
- * Short Description:** A text input field.
- State:** A dropdown menu with "New" selected and highlighted in green.
- Priority:** A dropdown menu showing "-- None --".
- Assignment Group:** A text input field with a search icon.

 A "Submit" button is located in the top right corner of the form.

Figure 3 — Employee raises issue intake form as delivered, showing Number (auto-generated, e.g. EMP0000010), Requester, Category, Subcategory, Short Description, State, Priority, and Assignment Group.

7. Screenshot Reference

Figure 2 shows the table definition as it exists in Studio, with the fields and controls used to generate a navigator module and default access rules automatically. Figure 3 shows the resulting intake form as a support agent or administrator would see it directly on the table — this is distinct from, but the basis for, the simplified Record Producer form employees use, which is built in Phase 4.

8. Choice List Values

8.1 Category

The categories configured in the working build reflect a simple, IT-service-oriented taxonomy:

- Software
- Hardware

- Network
- Access

8.2 Subcategory (dependent on Category)

Parent: Category	Dependent Subcategory Values
Hardware	Laptop, Server, Printer / Peripheral
Software	Application Installation, Application Error
Network	VPN, Wi-Fi / LAN Connectivity
Access	Forgot Password, Application Access Request, Role Change Request

NOTE: If your organization's issues span beyond IT (payroll, HR policy, general facilities), extend this choice list with additional top-level categories and matching subcategories before go-live. The dependent-choice mechanism in section 9 supports any number of category/subcategory pairs.

9. Table-Level Configuration

- Auditing — enabled, so field-level history is available for reporting and dispute resolution.
- Number format — auto-numbered, prefixed EMP.
- Default view — a dedicated form view with only the fields relevant to a self-service employee, separate from a fuller Agent view used by support staff.
- Dependent field configuration — Subcategory's dictionary entry has Depends on set to Category, with the mapping in section 8.2 configured under the Dependent choice list related list.

BEST PRACTICE: Model the Category / Subcategory relationship as a formal dependent choice list rather than a free-text field, so that reporting on issue types remains reliable as volume grows.

10. Access Control (ACL) Overview

Access to the Employee Issue table should be controlled at three levels, consistent with least-privilege principles:

1. Create — permitted for any authenticated employee, enforced primarily at the Record Producer layer described in Phase 4.
2. Read — an employee may read only records where Requester equals the logged-in user; agents with the appropriate assignment group may read all records within their group.
3. Write — restricted to assigned agents and administrators; the submitting employee should not be able to directly edit the record after submission, preserving the integrity of the original request.

Role	Read	Write
Requester (own records)	Yes	No, after submission
Assigned Agent (own group)	Yes	Yes
Administrator	Yes	Yes

11. Data Quality Considerations

- Enforce a minimum length on Description (for example 20 characters) so agents are not left with unusably vague submissions — implemented client-side in Phase 3 and mirrored server-side.

- Keep Short Description genuinely short; a 160-character limit forces a real summary rather than a restatement of the full description.
- Prefer reference fields (Requester, Department, Assignment Group) over free-text equivalents wherever the value should map to an existing platform record — this keeps reporting and routing reliable.
- Review choice list values periodically (see the Support and Maintenance guidance in the full project documentation) to retire unused categories and add new ones as workplace needs evolve.

11.1 Sample Record

The following illustrates a realistic, fully populated Employee Issue record, useful as a reference when building test data or explaining the table to a new team member:

Field	Example Value
Number	EMP0000010
Short Description	VPN keeps disconnecting during video calls
Description	Since Monday, my VPN connection drops every 10–15 minutes, especially during video calls. I've restarted my laptop twice; issue persists on both office Wi-Fi and home network.
Category	Network
Subcategory	VPN
Priority	Medium
Requester	Jane Employee
Department	Finance
State	New
Assignment Group	IT Infrastructure

12. Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
Subcategory shows all values regardless of Category	The dependent choice mapping was not configured, or Depends on was left blank on the dictionary entry.	Open the Subcategory dictionary entry, set Depends on to Category, and populate the Dependent choice list related list.
Auto Number does not generate on new records	The Number field was not configured to auto-number, or the number format was not saved before records were created.	Configure Auto Number on the field before any records exist; existing records will not be renumbered retroactively.
New fields do not appear on the employee-facing form	The field was added to the table but not to the view used by the Record Producer or portal form.	Add the field to the relevant view explicitly; a new dictionary entry is not automatically added to every existing view.
Attachments cannot be added by employees	The Attachment related list or field-level ACL restricts create access for non-agent roles.	Review the attachment ACL and confirm the employee role has create access on sys_attachment for this table.

Issue	Likely Cause	Resolution
Choice list order looks wrong on the form	The Sequence value on the choice records does not reflect the intended display order.	Edit each choice's Sequence field to control display order explicitly.

13. Validating Phase 2 Completion

- The Employee Issue table exists in the correct application scope and appears in Studio's Tables list.
- Every field in section 6 exists with the correct type and mandatory flag.
- Category and Subcategory are linked via a working dependent choice list.
- A test record can be created manually (via the table's default form) and saves without error.
- ACLs restrict a test employee user to only their own records, and allow a test agent user to see records in their assignment group.

14. Best Practices

BEST PRACTICE: Add auditing to any field an agent or employee might dispute later (state changes, resolution notes) — it costs little and pays for itself the first time a submission is questioned.

BEST PRACTICE: Resist the temptation to add every field stakeholders request. Each additional mandatory field increases abandonment on the Record Producer built in Phase 4 — derive what you can (Department, Assignment Group) rather than asking for it.

BEST PRACTICE: Build the dependent choice list (section 8) using the platform's native mechanism, not client-script-only filtering, so the dependency holds even if a future integration bypasses the portal form entirely.

14.1 Frequently Asked Questions

Should this table extend Task or stand alone?

Extending Task is attractive because it inherits familiar fields (Number, State, Priority, Assignment Group) and plugs into existing Task-based reporting and SLA tooling. The trade-off is inheriting many Task fields and behaviors that are not relevant to a simple employee issue, which can clutter the form and confuse agents. For a focused, standalone use case like this one, a lightweight custom table (as built here) is usually easier to reason about; extending Task is worth revisiting if SLA definitions or Task-based dashboards become a requirement later.

Can more categories be added after go-live?

Yes. Choice list values and their dependent mappings can be added at any time without a table structure change. New categories will not automatically have UI Policy or routing behavior, however — review Phase 3 and the routing logic referenced in Phase 4 whenever a new top-level category is introduced.

What happens to existing records if a field's type is changed later?

Changing a field's type after records exist can cause data loss or unreadable values (for example, converting a Choice field with existing values to a Reference field). Where a type change is unavoidable, plan a data migration script rather than editing the dictionary entry directly against a populated table.

15. Summary and Handoff to Phase 3

At the end of this phase, the Employee Issue table exists with a complete field list, dependent choice values, table-level settings, and a first-pass ACL model. This table is now ready to have dynamic, on-screen behavior layered onto it in Phase 3 — Creating UI Policies and Dependency — which governs what employees see and must complete based on the values they choose.

15.1 Phase 2 Deliverables Checklist

Deliverable	Status
Employee Issue table created in correct scope	Complete
All fields from section 6 added with correct types	Complete
Category / Subcategory dependent choice list configured	Complete
Table-level settings (auditing, numbering, views) applied	Complete
First-pass ACLs in place	Complete
Ready for Phase 3 UI Policies	Ready